

Описание алгоритма игрока

1. Общая идея алгоритма

Задача алгоритма - выбрать наилучший ход в игре крестики-нолики на поле 20x20. Алгоритм строит мысленное дерево возможных партий: из текущей позиции он перебирает несколько ходов за себя, затем за противника, затем снова за себя - и так далее на несколько ходов вперёд. Каждая конечная позиция этого дерева оценивается числом: чем оно больше - тем позиция выгоднее для нас.

Принцип выбора хода следующий: алгоритм предполагает, что противник также играет оптимально, то есть всегда выбирает ход, наиболее невыгодный для нас. Исходя из этого допущения, алгоритм находит такой ход, который гарантирует наилучший результат даже при идеальной игре соперника. Этот подход называется минимаксным поиском.

2. Методы оптимизации

Полный перебор всех возможных ходов на поле 20x20 невозможен - вариантов слишком много. Поэтому в алгоритме применяются четыре метода оптимизации, позволяющих укладываться в жёсткие временные ограничения:

- Альфа-бета отсечение: отсечение заведомо проигрышных ветвей дерева. Если в процессе анализа алгоритм обнаруживает, что текущий вариант не может превзойти уже найденный лучший результат, он прекращает рассматривать эту ветвь и переходит к следующей.
- Итеративное углубление: контроль времени хода. Алгоритм не пытается сразу просчитать максимальную глубину. Вместо этого он последовательно ищет лучший ход на глубине 1 ход вперёд, затем 2, затем 3 и так далее. Если время истекло до завершения очередного уровня, берётся результат с последней полностью завершённой глубины.
- Ограниченный перебор кандидатов: на каждом шаге анализируются не все пустые клетки поля, а только 10 наиболее перспективных кандидатов, отобранных эвристически. Наилучший из них всегда рассматривается первым, что существенно повышает эффективность альфа-бета отсечения.
- Таблица предвычисленных оценок (Look-Up Table): чтобы не вычислять оценку одной и той же комбинации клеток многократно, все возможные варианты (их $3^9 = 19683$) рассчитываются единожды при запуске программы и сохраняются в таблицу. Во время игры алгоритм просто обращается к нужной ячейке таблицы за готовым результатом.

3. Оценка клетки

Чтобы понять, насколько выгодно поставить знак в ту или иную клетку, алгоритм анализирует четыре линии из 9 клеток, проходящих через неё: горизонтальную, вертикальную и две диагонали. Анализируемая клетка всегда находится в центре каждой такой линии. В каждой линии подсчитывается длина непрерывной цепочки знаков одного игрока. Чем длиннее цепочка и чем свободнее её края (нет ли рядом знаков противника или стены), тем выше оценка. Итоговая ценность клетки складывается из оценок по всем четырём линиям отдельно для атакующего и защитного потенциала.

Длина цепочки	Оценка
5 и более в ряд	10 000 000
4 в ряд, оба конца свободны	1 000 000
4 в ряд, один конец свободен	100 000
3 в ряд, оба конца свободны	50 000
3 в ряд, один конец свободен	5 000
2 в ряд, оба конца свободны	500
2 в ряд, один конец свободен	50

Анализ окна из 9 клеток (по 4 клетки в каждую сторону от центра) позволяет алгоритму обнаруживать «вилки» - ситуации, когда у одного игрока одновременно возникают две или более опасных цепочки, требующие ответа. Такие позиции получают значительную надбавку к оценке.

4. Отбор кандидатов для анализа

Перед запуском полного перебора алгоритм сужает список рассматриваемых ходов до 10 наиболее перспективных. Это делается в два шага:

Шаг 1. Определение активной зоны. Алгоритм находит минимальный прямоугольник, охватывающий все уже сделанные ходы и препятствия, и расширяет его на 2 клетки в каждую сторону. Пустые клетки вне этой зоны не рассматриваются. На ранних этапах игры это существенно сокращает число кандидатов.

Шаг 2. Эвристическая оценка каждой клетки в зоне. Для каждой пустой клетки вычисляется ценность по формуле:

$$U = 10 \cdot A + 12 \cdot D + B$$

где A - атакующий потенциал (насколько эта клетка помогает нам построить цепочку), D - защитный потенциал (насколько она мешает аналогичным планам противника), B - бонус за близость к центру доски. Коэффициент D намеренно взят чуть больше: защита от вилок противника приоритетнее атаки. Если в позиции обнаруживается двойная угроза (вилка), клетка получает дополнительную надбавку.

После вычисления ценностей все клетки сортируются по убыванию, и в дальнейшую работу передаются только 10 лучших. Лучший кандидат помещается первым. Это позволяет альфа-бета отсечению отсекал большую часть дерева уже на ранних ветвях.

5. Порядок работы алгоритма

При каждом ходе алгоритм выполняет следующую последовательность действий:

1. Отбираются 10 кандидатов для текущей позиции.
2. Запускается итеративное углубление: алгоритм оценивает лучший ход на глубину 1, затем 2, и так далее до максимума (10 ходов вперёд) или до истечения времени.
3. На каждой глубине для каждого кандидата строится поддерево возможных продолжений. На каждом уровне поддерева снова отбираются 10 лучших кандидатов для текущего игрока.
4. Если алгоритм достигает заданной глубины, позиция оценивается итоговой эвристикой, не погружаясь глубже.
5. По завершении каждого уровня глубины лучший найденный ход фиксируется. Если время истекает посередине уровня, берётся результат предыдущего завершённого уровня.
6. Возвращается ход с наибольшей гарантированной оценкой.

Таким образом, алгоритм всегда успевает вернуть ответ в срок, при этом используя доступное время максимально эффективно: чем больше времени, тем глубже и точнее анализ.